Confidential Inference Proof-of-Concept: Progress Report

Privacy-preserving edge gateway for real-time audio deepfake detection — OP-TEE/TrustZone confidential inference PoC using QEMU emulation.

---

1. Context and Objective

The overall thesis targets a privacy-preserving edge gateway for real-time audio deepfake detection in residential telephony, using a trained RawTFNet_small model as the detection backbone and a Raspberry Pi 4 as the intended deployment target. Since no physical Raspberry Pi 4 hardware is available during development, this phase of the work establishes a proof-of-concept (PoC) for **confidential inference** — running the trained model's computation inside an isolated, hardware-backed trusted execution environment (TEE) so that the model weights and inference logic are protected from a compromised or malicious host operating system. The PoC is being developed and validated on an emulated ARM environment before any hardware-specific integration work begins.

2. Methodology and Justification of Key Decisions

2.1 Platform choice: QEMU ARMv8 “virt” reference board, not a Raspberry Pi 4 emulator

No official or maintained emulator exists that boots “a Raspberry Pi 4” with a working TrustZone configuration. Broadcom has not published the secure-boot and TrustZone configuration details for the BCM2711 SoC used in the Pi 4, so the OP-TEE project has no official port to that board, emulated or physical. The methodologically correct and standard substitute — used throughout the OP-TEE research and tooling ecosystem — is the **QEMU “virt” Armv8-A reference platform** (the `qemu_v8` manifest in the official OP-TEE build tooling). It shares the same CPU architecture family as the Pi 4 (ARMv8-A / AArch64) and implements the same TrustZone secure/non-secure world security model, while being fully and officially supported by OP-TEE. This is stated explicitly here so that the choice reads as a deliberate, justified scope decision in the thesis rather than an unexplained substitution.

2.2 Development host: VirtualBox Ubuntu VM with bridged networking, not WSL2

OP-TEE's build system (the `repo` multi-repository tool, Buildroot, device-tree compilation, cross-toolchains) requires a native Linux environment and does not build reliably under Windows compatibility layers. A Linux VM was chosen over WSL2 specifically because the end goal of this phase includes receiving live audio from an Asterisk server, and Asterisk's SIP/RTP traffic requires a stable, directly reachable network address. WSL2's default NAT layer and internal IP volatility are a known source of unreliability for real-time media traffic. A VM with a **bridged network adapter** gives the Linux environment a stable IP address on the same network as any calling client, while still allowing Windows and Linux to run side by side without dual-boot reboots.

2.3 Model export path: PyTorch → ONNX → TVM, not hand-ported C

A trusted application (TA) running inside OP-TEE executes in a minimal, single-threaded environment with a restricted C library and a small fixed secure-memory heap; it cannot host a full deep learning framework such as PyTorch or ONNX Runtime. Rather than manually re-implementing RawTFNet's

layers in C (high effort, high risk of numerical divergence from the trained model), the model is exported to **ONNX** as a framework-neutral graph representation, then compiled using **Apache TVM** into ahead-of-time, dependency-free code suitable for embedding inside a TA. Correctness is verified numerically at every conversion step against the original PyTorch output, so that any discrepancy is caught at the specific stage that introduced it rather than surfacing later inside the TEE, where debugging is significantly harder.

3. Work Completed

Stage	What was done	Result / evidence
M1	Built OP-TEE OS, TF-A, Linux kernel, and Buildroot rootfs for the QEMU Armv8-A *virt* platform from the official OP-TEE manifest (`qemu_v8.xml`); booted the environment and ran the official OP-TEE test suite.	System booted with a working secure-world / normal-world split. `xtest` completed with **0 failures across 41,482 subtests and 145 test cases**.
M2	Built and ran the official `hello_world` TA/CA example to validate the client-application-to-trusted-application invocation pattern that the model inference TA will later reuse.	Client application successfully opened a session with the trusted application and received a correct computed result back (`TA incremented value to 43`).
M3	Exported the trained RawTFNet_small checkpoint (`Best_RawTFNet_16.pth`) to ONNX; verified the ONNX output against the original PyTorch output; compiled the ONNX graph with Apache TVM (Relax IR, LLVM JIT target) on the Ubuntu development VM; verified the TVM-executed output against both prior stages on the same test utterance.	All three representations of the model (PyTorch, ONNX, TVM/Relax) produced the **same score to four decimal places** (–5.6216 / –5.621588) on a held-out ASVspoof2021 LA evaluation utterance, confirming the exported and compiled model is numerically faithful to the originally trained model.

Verification commands and representative output are reproduced below for completeness.

```
PyTorch score: -5.6216
ONNX score: -5.6216
Difference: 0.000000

TVM (relax/llvm) score: -5.621588
```

4. Limitations

The following limitations apply to the current PoC and are recommended for explicit statement in the thesis's Limitations / Threats to Validity section.

Limitation	Explanation
No physical target hardware	All results were obtained on the QEMU Armv8-A *virt* reference platform rather than a physical Raspberry Pi 4. This validates the software architecture and the TrustZone isolation model but not device-specific behaviour (e.g. real boot chain, peripheral timing, thermal/performance characteristics of the BCM2711 SoC).
No hardware root of trust	QEMU's secure world is a software CPU-mode partition with no fused keys, no physical secure boot chain, and no tamper resistance. The PoC therefore demonstrates the correctness of the confidential-inference software design, not a hardware-attested security guarantee. Hardware-backed attestation is out of scope for the PoC and is identified as future work.
Model framework cannot run inside the TA directly	PyTorch and ONNX Runtime cannot execute inside OP-TEE's restricted TA environment (no dynamic framework runtime, small fixed heap, single-threaded execution). The model must instead be compiled ahead-of-time to dependency-free code via TVM before it can be linked into a trusted application — this compilation step (M4) is still in progress at the time of writing.
Toolchain instability	The installed Apache TVM release (0.25.0.post1) is a very recent build in which the classical Relay API has been fully replaced by a new Relax API, and several supporting modules (tensor/array handling, ONNX frontend import paths) differ from most published TVM tutorials and documentation. Multiple import-path and API-location errors were encountered and resolved empirically during this phase; this is reported as a practical toolchain risk for reproducibility, and pinning a specific, documented TVM version is recommended before final thesis submission.
Feature extraction scope (planned, not yet built)	In the planned TA design, only the model's forward pass will run inside the trusted world; audio framing and feature extraction will remain in the untrusted normal world for this PoC. This is a deliberate, stated scope boundary — the property being demonstrated is that model weights and computation are protected from a compromised host, not that the entire audio pipeline is confidential.

5. Next Steps

Stage	Planned work
M4	Recompile the verified ONNX/Relax model using TVM's C code-generation target (rather than the LLVM JIT target used for verification in M3) to produce standalone, dependency-free C source suitable for compilation inside the OP-TEE TA build environment.
M5	Create a new trusted application, structured after the verified `hello_world` example (M2), exposing a single command (e.g. `TA_CMD_RUN_INFERENCE`) that runs one forward pass of the compiled model on a fixed-length input buffer and returns a spoof/bonafide score.
M6	Write a client application that sends one static, known test vector to the TA and confirms the returned score matches the PyTorch/ONNX/TVM reference score obtained in M3, closing the correctness loop from trained model through to in-TEE inference.
M7	Integrate the client application with the Asterisk rolling audio buffer so that live call audio is windowed, fed to the trusted application, and a spoof/bonafide decision is produced during an active call — the end-to-end live PoC.

Each milestone above is independently demonstrable and reportable, which allows the thesis to present verified, working results up to whichever milestone is reached by the submission deadline.